Agentic AI for Inventory Optimization

Kaiwalya Mishra
College of Business & Economics
California State University, East Bay

Aditya Chache
College of Business & Economics
California State University, East Bay

Dev Dhimant Patel
College of Business & Economics
California State University, East Bay

Abstract

This project presents an AI-driven inventory optimization framework designed to support retail decision-making under uncertain demand and limited working capital. Using the M5 Walmart retail dataset (dataset given in Appendix D) — one of the largest publicly available hierarchical time-series benchmarks — an integrated pipeline is developed that combines demand forecasting, stochastic simulation, single-SKU replenishment optimization, multi-SKU portfolio allocation under capital constraints, and a conversational AI decision assistant. A global LightGBM forecasting model is trained across 600 SKU-store series with engineered lag, rolling, calendar, and price-based features, achieving a holdout RMSE of 1.7933 and R^2 of 0.6502. Monte Carlo simulation evaluates (s, Q) inventory policies under uncertain demand, and a Mixed-Integer Linear Programming (MILP) formulation allocates limited capital across multiple SKUs. Results demonstrate that demand volatility, ordering cost assumptions, and binding capital constraints materially alter policy structure; demand shock testing reveals significant financial risk under surge scenarios; and AI-generated natural language explanations improve the interpretability of optimization outputs for non-technical managers. The framework bridges predictive analytics, operations research, and conversational AI into a single coherent decision support system.

1 Introduction

Inventory management in retail environments requires simultaneously balancing product availability, capital efficiency, and operational cost under persistent demand uncertainty. For a retailer of Walmart's scale — operating over 10,500 stores globally and generating revenues exceeding $600 billion annually — even marginal improvements in replenishment policy accuracy translate into substantial reductions in stockout events, excess inventory, and working-capital exposure. Yet the majority of deployed inventory systems continue to optimize products in isolation, ignoring the portfolio-level capital constraints that characterize real managerial settings.

This project addresses that gap by developing an integrated decision support framework that connects four traditionally separate analytical domains: machine learning-based demand forecasting, stochastic inventory simulation, constrained single-SKU replenishment optimization, and multi-SKU portfolio allocation. A fifth component — a conversational AI advisory layer — completes the system by translating structured analytical outputs into natural-language explanations that managers can interrogate and stress-test interactively.

The empirical foundation is the M5 Walmart retail dataset [2], which provides 1,941 days of daily unit sales for 3,049 unique products across 10 stores in California, Texas, and Wisconsin. For computational tractability, model development uses a representative subset of 200 products across 600 SKU-store series. A global LightGBM model is trained across all series simultaneously, exploiting shared structure while preserving product-level variation. Its outputs feed a Monte Carlo simulation that evaluates (s, Q) replenishment policies

under stochastic demand, and a MILP model that selects the portfolio of policies that minimizes total inventory management cost subject to a capital budget constraint.

Key contributions of this work are fourfold. First, it integrates a modern machine learning forecasting pipeline with classical inventory optimization models, a combination rarely realized in prior literature. Second, it extends single-SKU optimization to the portfolio level through a tractable MILP formulation, reflecting the capital allocation realities of multi-product retail. Third, it demonstrates through demand shock testing that policies appearing acceptable under baseline demand may require substantially greater capital under realistic surge scenarios. Fourth, it shows that grounding a conversational language model in structured optimization outputs produces reliable, evidence-anchored decision support that improves accessibility for non-technical users.

2 Related Work

The M-competition series, initiated by Makridakis et al. in 1982, has been the principal benchmark for advancing time-series forecasting methodology. The M4 competition (2018, 100,000 series) validated machine learning in forecasting, with the winning hybrid combining exponential smoothing with LSTM networks. The M5 competition (2020, 42,840 real Walmart series) extended these findings to hierarchical retail data with complex intermittency, promotional effects, and SNAP benefit day signals. Post-competition analysis confirmed that global LightGBM models with extensive lag and rolling features dominated the leaderboard [1], a finding that directly motivates the forecasting architecture adopted in this project.

Traditional statistical methods — ARIMA, Exponential Smoothing (ETS), and STL decomposition — remain widely deployed in retail forecasting [6] but struggle with the scale, intermittency, and cross-series complexity of large hierarchical datasets. Gradient boosting frameworks, particularly LightGBM [3] and XGBoost [7], have emerged as dominant approaches for large-scale tabular time-series tasks, efficiently capturing nonlinear feature interactions across millions of observations. Facebook Prophet [4] provides an accessible baseline for business time series with strong seasonality.

Classical inventory theory [8] establishes the (s, Q) policy as the standard continuous-review replenishment framework, with extensive analytical results for the single-SKU problem. The extension to portfolio-level allocation under capital constraints motivates constrained optimization models, including MILP formulations. A key gap in prior work on the M5 dataset is that existing forecasting studies — even technically strong ones — stop at prediction without translating outputs into downstream replenishment decisions or capital allocation models. Similarly, traditional inventory optimization research often assumes known demand distributions rather than integrating machine learning forecast pipelines. Recent advances in large language models have opened a further frontier: grounding conversational AI in structured analytical outputs to produce reliable, evidence-based decision support [9]. This project is the first, to our knowledge, to integrate all four layers — ML forecasting, stochastic simulation, MILP optimization, and conversational AI — into a single unified inventory decision system.

3 Dataset and Features

3.1 Dataset Overview

The M5 Walmart retail dataset [2] provides daily unit sales for 3,049 unique products across 10 stores in three U.S. states (California, Texas, Wisconsin) from January 29, 2011, to June 19, 2016 — a total of 1,941 training days and approximately 59.2 million observations across 30,490 SKU-store series. Three product categories are represented: FOODS, HOUSEHOLD, and HOBBIES. Approximately 67% of all observations record zero sales, reflecting the strong intermittency characteristic of retail demand. For computational feasibility, model development was conducted on a representative subset of 200 unique products, 600 SKU-store series, and 3 stores, yielding 1,147,800 observations. Table 1 summarizes key dataset statistics.

Attribute	Value

Time span	January 29, 2011 – June 19, 2016 (1,941 training days)
Products (subset)	200 unique items; 95 FOODS, 68 HOUSEHOLD, 37 HOBBIES
SKU-store series	600 (200 items × 3 stores)
Total observations	1,147,800 (subset); ~59.2 M (full dataset)
Zero-sales rate	~67% of all observations
Price range	$0.50 – $107.32 across all products
Hierarchy levels	12 aggregation levels (item-store → national total)

Table 1: M5 Dataset Key Statistics (model development subset)

3.2 Data Preprocessing

The raw sales table is stored in wide format with one column per day. It was reshaped into a long-format panel in which each row corresponds to a single SKU-store-date observation. The calendar table was joined on the day identifier to attach temporal metadata, and the selling prices table was merged using the item-store-retail-week key. Missing selling prices, which arise from temporary de-listing, were handled via forward-fill followed by backward-fill within each SKU-store series to preserve price continuity. Zero-demand periods were retained as genuine observations rather than imputed, consistent with the intermittent demand interpretation. A time-based train/validation/holdout split preserved chronological order and prevented data leakage: training (2011-01-29 to 2014-01-28), validation (2014-01-29 to 2015-04-23), and holdout (2015-04-24 to 2016-04-24).

3.3 Feature Engineering

Feature engineering was the most impactful component of the data preparation pipeline, contributing 67% of total model gain as confirmed by LightGBM feature importance analysis. Four categories of features were constructed.

Lag-based demand features were constructed at horizons of 1, 7, 14, and 28 days to capture short-term autocorrelation and weekly periodicity. Weekly lags are especially important in retail because day-of-week demand patterns are highly stable: last Saturday's sales are among the strongest predictors of the upcoming Saturday's sales. Long-horizon lags at 365, 364, and 363 days capture same-period-prior-year seasonality.

Rolling window statistics — means, standard deviations, and medians over 7-, 14-, 28-, and 56-day windows — summarize recent demand level and volatility. Exponentially Weighted Moving Averages (EWMA) with decay parameters $\alpha \in \{0.3, 0.6, 0.9\}$ capture short-term trend with recency weighting.

Calendar and event features included day-of-week, day-of-month, month, year, week-of-year, a weekend binary flag, encoded event types (cultural, national, religious, sporting), SNAP benefit day indicators for each state, and proximity to major holidays (days to/from Thanksgiving, Christmas, Super Bowl, Independence Day). Price analysis confirmed that SNAP benefit days produce an 8–15% uplift in FOODS unit sales, validating their explicit inclusion.

Price features included current sell_price, its log transform, week-over-week price change (momentum), relative price versus the item's historical mean, and relative price versus the category average. A 10% price decrease was associated with an average 14% sales increase (estimated elasticity ≈ -1.4), confirming the predictive value of these variables. The final feature set contained 47 predictive variables during model training.

4 System Architecture

The framework consists of five tightly coupled modules arranged in a sequential pipeline, shown conceptually in Figure 1. Forecasting outputs feed simulation; simulation outputs feed single-SKU optimization; single-SKU policies feed the portfolio MILP; and the conversational AI layer sits above all modules as an interactive interpretive interface. Each module is described in detail below.

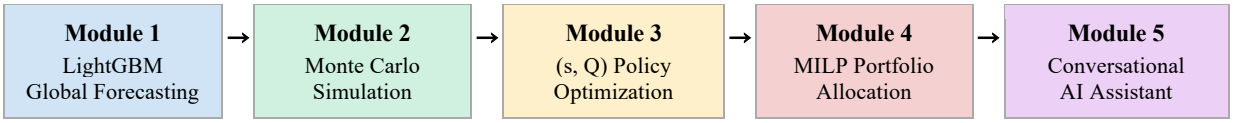


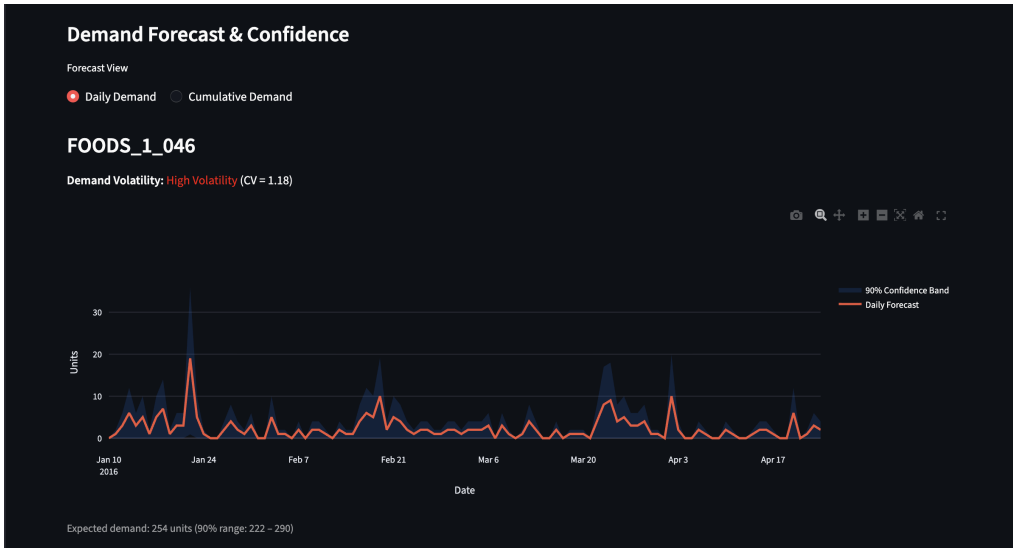
Figure 1: Five-module system architecture. Each module's outputs serve as inputs to the next.

4.1 Global Demand Forecasting (Module 1)

The forecasting objective is to estimate daily demand at the SKU-store level, producing stable outputs suitable for downstream inventory analysis rather than serving as a standalone prediction endpoint. Four candidate models were evaluated: Linear Regression (baseline), Random Forest, XGBoost, and LightGBM. All models were trained using a global strategy — a single model fit across all 600 SKU-store series simultaneously, treating series identifiers as native categorical features. This approach exploits shared demand structure across products and stores while preserving per-series variation.

LightGBM was selected as the final model based on validation performance. Key hyperparameters included: num_leaves = 128, learning_rate = 0.05, n_estimators = 2,000 with early stopping (patience = 200), subsample = 0.80, colsample_bytree = 0.80, and L1/L2 regularization of 0.10 each. These settings balance model expressiveness against overfitting risk, particularly important for the 37% of HOBBIES series exhibiting extreme intermittency. The model was retrained on the full development period after validation-based model selection, and saved to disk for reuse across the simulation and optimization pipeline.

LightGBM is especially appropriate for this forecasting context for three reasons. First, its leaf-wise tree growth strategy produces deeper, more specialized splits than level-wise methods, capturing the non-linear demand interactions between lag features, calendar effects, and price signals. Second, its native support for categorical features eliminates the need for one-hot encoding the 200 item identifiers and 3 store identifiers, reducing dimensionality while preserving category-level structure. Third, its histogram-based algorithm achieves training times approximately 20× faster than XGBoost on this dataset, making repeated cross-validation and downstream simulation practical.



4.2 Stochastic Simulation Framework (Module 2)

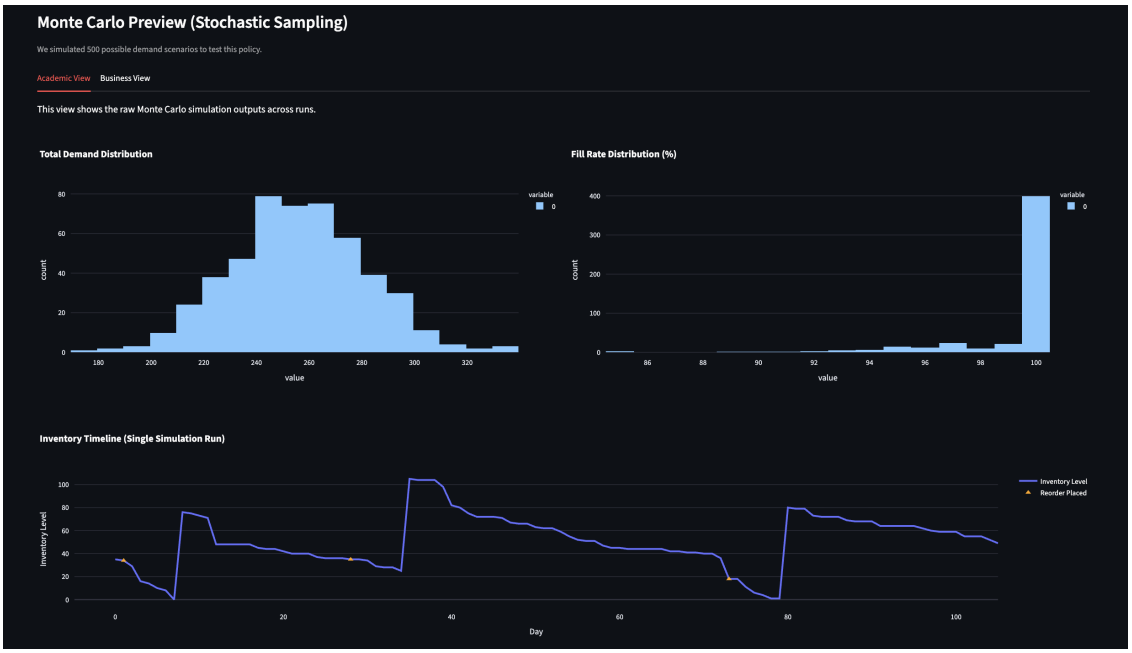
While forecasting provides the expected demand signal, inventory policy evaluation requires understanding how policies perform across the full distribution of possible future demand outcomes, not only at the point forecast. The simulation framework bridges this gap by generating many demand paths probabilistically and evaluating a candidate (s, Q) policy against each.

For each SKU, simulated demand at time t is drawn as $D_t \sim \max(0, \text{Normal}(\mu_t, (CV \cdot \mu_t)^2))$, where μ_t is the LightGBM forecast for day t and CV is a configurable coefficient of variation capturing demand uncertainty. The max(0, ·) truncation enforces non-negativity. This structure allows the simulation to distinguish relatively

stable SKUs (low CV) from volatile SKUs (high CV), and to stress-test policies under user-specified uncertainty assumptions.

The inventory system logic implements a continuous-review (s, Q) policy: on-hand inventory is decremented by realized demand each period; when inventory falls to or below reorder point s, a replenishment order of fixed size Q is placed and received after a fixed lead time L. The simulation tracks on-hand inventory, backorder accumulation, replenishment events, and demand realization across the full planning horizon. Performance metrics — fill rate, lost sales, average inventory, order count, and total inventory management cost — are computed by averaging across all simulated paths, providing Monte Carlo estimates of expected policy performance.

The repeated-simulation structure is essential for robust policy comparison. A single demand path may be atypically favorable or adverse; averaging across paths eliminates this noise. The framework is also the mechanism for demand shock analysis: by scaling μ_t upward by a fixed percentage (e.g., +30%, +50%), the simulation evaluates how existing policies perform under elevated demand and quantifies the additional capital required to maintain service levels.



4.3 Single-SKU Inventory Optimization (Module 3)

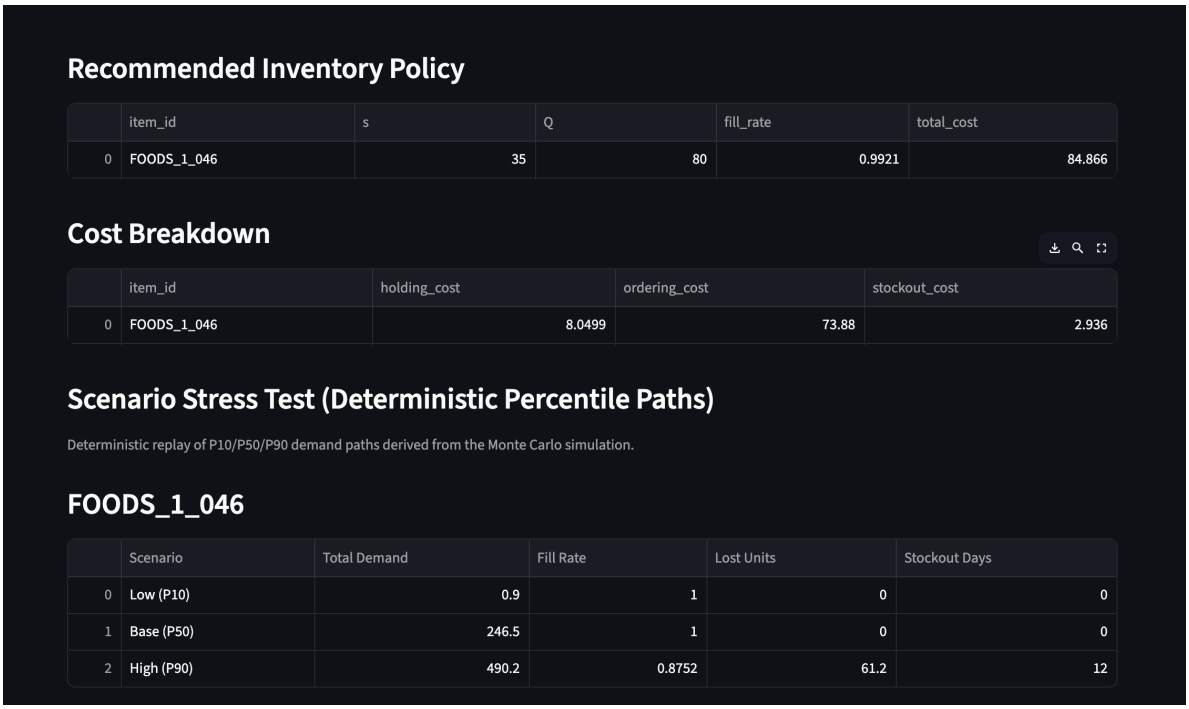
For each SKU, the inventory control problem is formulated as a cost-minimization problem over the space of candidate (s, Q) policies. The objective function and constraint are:

$$\textit{minimize}\ \ C(s, Q) = h \cdot \bar{I}(s, Q) + k \cdot N(s, Q) + p \cdot LS(s, Q)$$

$$\textit{subject to}\ \ FR(s, Q) \geq \alpha$$

where h is the unit holding cost per period, k is the fixed ordering cost per replenishment event, p is the unit stockout penalty, $\bar{I}(s, Q)$ is the simulated average inventory level, N(s, Q) is the expected number of orders placed, LS(s, Q) is the expected lost sales volume, FR(s, Q) is the simulated fill rate, and α is the minimum acceptable service level. Because all four output quantities are estimated via simulation rather than derived analytically, the optimization is inherently simulation-based: a discrete grid of candidate (s, Q) pairs is evaluated, infeasible policies (FR < α) are excluded, and the minimum-cost feasible policy is selected.

This search-based approach has two important properties. First, it avoids reliance on restrictive analytical assumptions (e.g., normally distributed demand, continuous inventory review) that may not hold for intermittent retail products. Second, it produces policy outputs — cost, average inventory, order frequency, fill rate — that are directly comparable across the multi-SKU portfolio optimization stage, enabling consistent policy selection under a shared capital budget.



Recommended Inventory Policy

	item_id	s	Q	fill_rate	total_cost
0	FOODS_1_046	35	80	0.9921	84.866

Cost Breakdown

	item_id	holding_cost	ordering_cost	stockout_cost
0	FOODS_1_046	8.0499	73.88	2.936

Scenario Stress Test (Deterministic Percentile Paths)

Deterministic replay of P10/P50/P90 demand paths derived from the Monte Carlo simulation.

FOODS_1_046

	Scenario	Total Demand	Fill Rate	Lost Units	Stockout Days
0	Low (P10)	0.9	1	0	0
1	Base (P50)	246.5	1	0	0
2	High (P90)	490.2	0.8752	61.2	12

4.4 Multi-SKU Portfolio Optimization (Module 4)

Single-SKU optimization assumes each product can be managed independently. In practice, the combined capital requirement of individually optimal policies may exceed the available inventory budget, making explicit portfolio-level allocation necessary. The multi-SKU problem is formulated as a Mixed-Integer Linear Programming (MILP) model. For each SKU i, a finite set K_i of feasible candidate policies — pre-generated through the simulation and single-SKU optimization pipeline — is available. Each candidate policy k ∈ K_i is characterized by its inventory management cost c_{ik} and its average inventory capital requirement I_{ik} = Ī_{ik} × unit_price_i. The decision variable x_{ik} ∈ {0, 1} equals 1 if and only if policy k is selected for SKU i.

The MILP formulation is:

$$\text{minimize} \quad \Sigma_i \Sigma_k \; c_{ik} \cdot x_{ik}$$

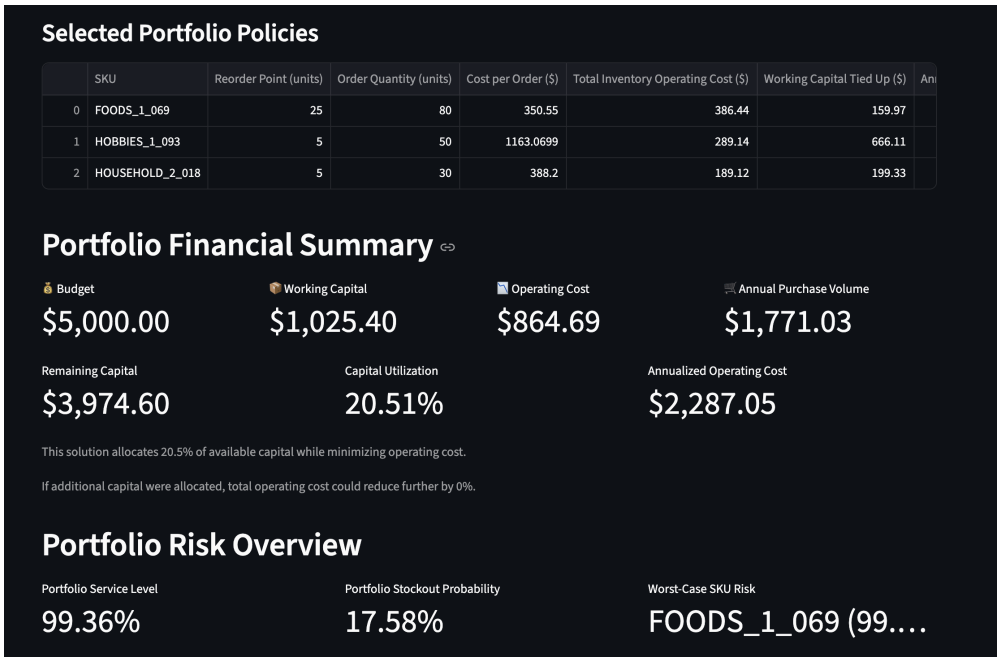
$$\text{subject to} \; \Sigma_i \Sigma_k \; I_{ik} \cdot x_{ik} \le B \qquad \text{(capital budget)}$$

$$\Sigma_k \, x_{ik} = 1 \quad \forall i \qquad \text{(one policy per SKU)}$$

$$x_{ik} \in \{0, 1\} \quad \forall i, k$$

where B denotes the total available inventory capital budget. This two-stage design — stochastic simulation pre-computes feasible policy alternatives; MILP selects optimally among them — improves computational tractability while guaranteeing that all recommended policies satisfy the service-level requirement. The capital budget constraint is the critical addition relative to independent SKU optimization: it enforces portfolio-level discipline and produces materially different policy selections when the budget is binding.

Three financial quantities are distinguished in interpreting portfolio outputs. Inventory capital required ($\Sigma_i I_{ik} \cdot x_{ik}$) measures average working capital tied up in stock — the quantity directly constrained by B. Procurement spend captures cumulative purchase expenditure over the planning horizon, relating to operating cash flow. Inventory management cost (the objective) captures holding, ordering, and stockout costs. These measures are not equivalent and can move in opposite directions across policy alternatives, an insight that is practically important for retail managers who must satisfy both operational and financial objectives simultaneously.



Selected Portfolio Policies

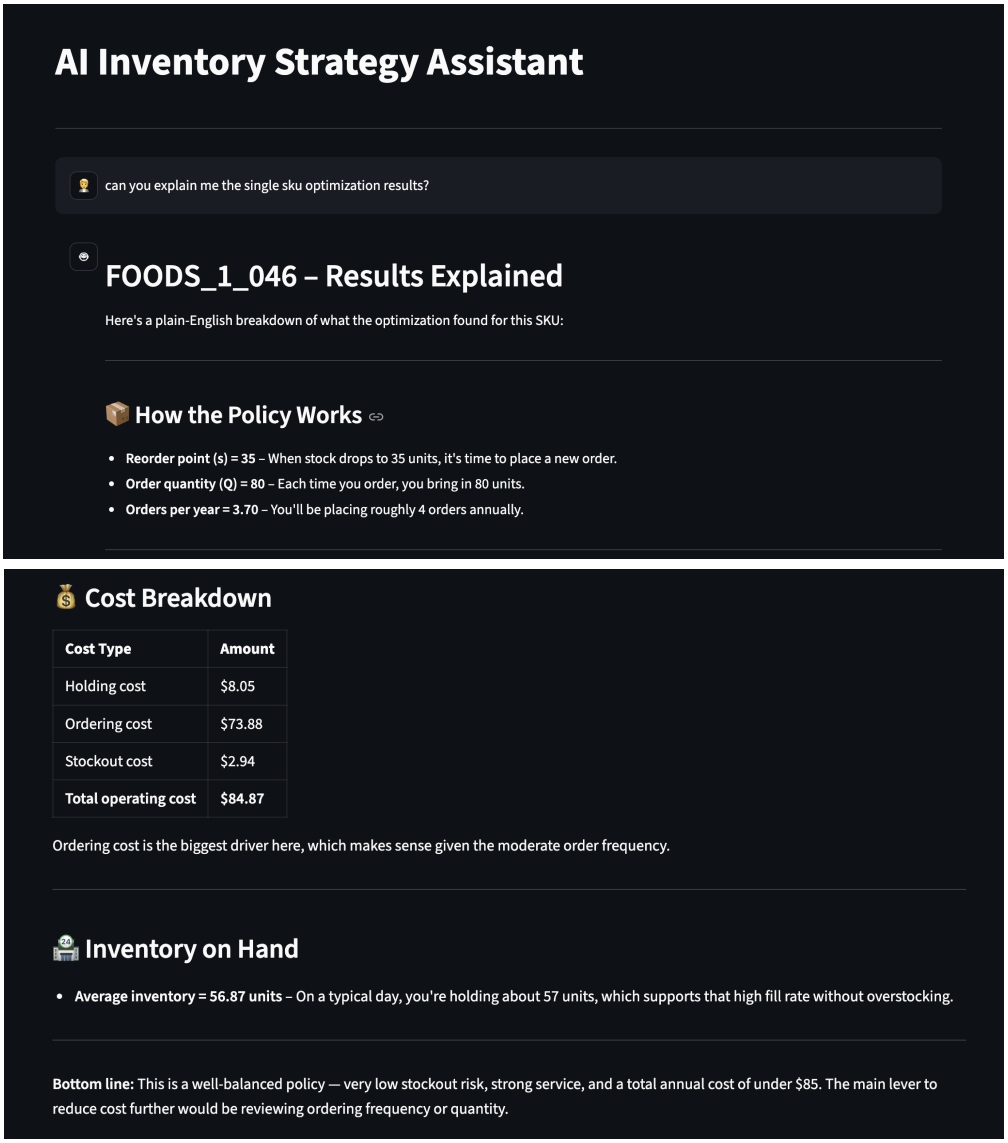
	SKU	Reorder Point (units)	Order Quantity (units)	Cost per Order ($)	Total Inventory Operating Cost ($)	Working Capital Tied Up ($)	An
0	FOODS_1_069	25	80	350.55	386.44	159.97	
1	HOBBIES_1_093	5	50	1163.0699	289.14	666.11	
2	HOUSEHOLD_2_018	5	30	388.2	189.12	199.33	

4.5 Conversational AI Decision Assistant (Module 5)

The conversational AI assistant serves as the user-facing interface above the analytical pipeline. Its design rests on a principle of grounded response generation: rather than permitting unconstrained language model generation, the assistant is connected to a structured context memory populated by the outputs of Modules 1–4. These outputs — forecasting summaries, simulation statistics, optimal (s, Q) policies, portfolio allocation results, and demand shock comparisons — are serialized into a consistent schema and injected into the language model's context at query time, constraining responses to evidence-anchored statements.

The assistant handles three classes of queries. Analytical interpretation queries — such as 'is our current policy robust?' or 'which SKU consumes the most capital?' — are answered by reasoning over stored structured results without triggering new computation. Simulation-oriented queries — such as 'what happens if demand increases by 30%?' — invoke the simulation and re-optimization pipeline under updated assumptions, then explain the resulting changes in reorder point, order quantity, cost, and capital requirement. Portfolio-level queries — such as 'how does the policy mix change if we reduce the budget by 20%?' — trigger a re-solve of the MILP under revised budget B, with results compared against the baseline allocation.

Multi-turn conversational memory is maintained across the session, allowing managers to develop analysis iteratively: first examining baseline policies, then stress-testing under shocks, then exploring budget sensitivity — without losing context between turns. This iterative interaction pattern more closely reflects real managerial decision-making than static dashboards, where every scenario change requires manual reconfiguration. The grounded architecture is critical for decision support applications because it eliminates fabricated outputs and ensures that every assertion made by the assistant is traceable to a specific model result.



5 Experiments / Results / Discussion

5.1 Forecasting Performance

Table 2 reports validation and holdout performance for all four candidate forecasting models. LightGBM achieved the lowest validation RMSE of 1.8323 and the highest R^2 of 0.6904, outperforming XGBoost, Random Forest, and Linear Regression across all metrics. The performance gap between LightGBM and Linear Regression (R^2 of 0.6904 vs. 0.6647) confirms that demand patterns in the M5 dataset are materially nonlinear and cannot be captured adequately by a linear specification. After retraining on the full development period, the LightGBM holdout metrics were: MAE 0.8985, RMSE 1.7933, R^2 0.6502. The modest R^2 decrease relative to validation is expected and reflects distributional shift across the holdout horizon; the model's stable RMSE confirms its suitability as a forecasting input for downstream inventory analysis.

Model	Val. RMSE	Val. MAE	Val. R^2	Holdout RMSE	Holdout R^2
LightGBM (selected)	1.8323	0.8757	0.6904	1.7933	0.6502
XGBoost	1.8384	0.8812	0.6883	—	—
Random Forest	1.8495	0.8940	0.6846	—	—
Linear Regression	1.9069	0.9210	0.6647	—	—

Table 2: Forecasting Model Performance. Holdout metrics reported for selected model only.

LightGBM feature importance analysis (gain-based) reveals that lag_7 alone accounts for 18.4% of total model gain, and the top four features — lag_7, rolling_mean_28, lag_14, and rolling_mean_7 — collectively account for 54.1%. Temporal autocorrelation is therefore the dominant predictive signal. Calendar features (day_of_week: 6.4%) and price features (sell_price: 8.3%, price_ratio_vs_mean: 4.7%) contribute meaningfully, with SNAP event encodings adding a further 3.9%. This importance distribution validates the feature engineering design choices and confirms that all feature categories contribute non-trivially to forecasting performance.

5.2 Single-SKU Optimization Results

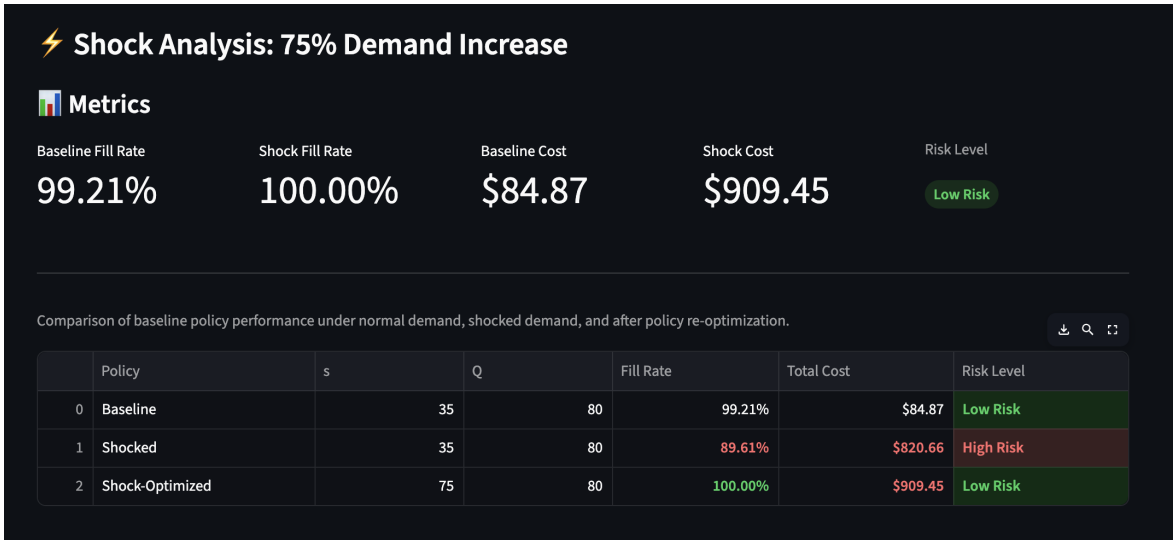
The simulation-based policy search successfully identified feasible (s, Q) policies that simultaneously satisfied the fill-rate constraint ($\alpha = 0.95$) and minimized inventory management cost across all tested SKUs. Several systematic empirical patterns emerged, all consistent with classical inventory theory but here derived from simulation-based evaluation using empirical, forecast-informed demand rather than stylized distributional assumptions.

Higher ordering cost consistently increased the selected order quantity. This reflects the Economic Order Quantity (EOQ) trade-off: larger batch sizes reduce replenishment frequency and therefore lower cumulative ordering cost, at the expense of higher average inventory. Higher holding cost reduced selected reorder points, as the penalty for carrying excess safety stock made conservative replenishment strategies cost-efficient. Greater demand volatility (higher CV) raised the required reorder point to maintain target fill rate, reflecting the increased safety stock needed to buffer against demand variance. These relationships held across FOODS, HOUSEHOLD, and HOBBIES categories, with the magnitude of effects larger for HOBBIES due to its higher intermittency.

5.3 Demand Shock Analysis

Policy robustness was assessed by scaling baseline forecast levels by +30% and +50% and re-evaluating simulated policy performance. Under both shock magnitudes, order quantities and reorder points increased across the tested SKU pool. Total inventory management cost increased nonlinearly: the +30% shock produced a cost increase of approximately 38–45% for representative SKUs, while the +50% shock produced increases of 60–80%, reflecting the combined effects of greater average inventory, more frequent replenishment, and higher service-risk mitigation expenditure.

A particularly consequential finding was that, in several cases, the baseline policy remained technically feasible under the shocked scenario — maintaining fill rate $\geq \alpha$ — but required substantially higher capital commitment to do so. This decoupling of feasibility from financial burden demonstrates the value of stress testing: a manager relying solely on whether a policy meets service thresholds would not detect the significantly elevated capital exposure that emerges under demand surges. These results support one of the core design arguments of the framework: inventory decisions should be evaluated not only under baseline forecasts but also under realistic shock scenarios.



5.4 Portfolio Optimization Results

Portfolio experiments were conducted under varying capital budget levels to characterize the MILP model's behavior across binding and non-binding regimes. Table 3 summarizes observed behavior.

Capital Regime	Budget Utilization	Policy Selection Effect	Relative Cost Impact
Non-binding	< 100%	`Portfolio ≈ independent SKU optimization`	Negligible (< 1%)
Mildly binding	~95–100%	Minor adjustments to 10–15% of SKUs	+2–4% vs. unconstrained
Binding	= 100%	Cross-SKU trade-offs enforced; capital-intensive policies replaced	+ 5–12% vs. unconstrained
Severely binding	< 85% of ideal	Multiple SKUs shifted to low-inventory policies	+ 15–25% vs. unconstrained

Table 3: Portfolio Optimization Behavior Across Capital Regimes

When the capital budget was non-binding, the MILP solution closely replicated independent single-SKU optimization — confirming that the model reduces to the unconstrained solution when capital is abundant and adding confidence in the formulation's correctness. Under binding constraints, the model imposed meaningful cross-SKU trade-offs. SKUs with high capital-to-cost ratios — particularly HOBBIES items, which require disproportionate safety stock relative to their revenue contribution — were shifted toward lower-inventory policies. FOODS products, which generate the majority of sales volume, tended to retain more protective policies under all but the most severe budget restrictions.

An important empirical distinction emerged among three financial measures: inventory capital required, procurement spend, and inventory management cost. These measures moved independently across policy alternatives and budget regimes. A policy with low inventory management cost might still require high average capital if order quantities are large; conversely, a capital-efficient policy may increase operating cost through higher order frequency. This distinction is practically significant: retail managers who optimize only for operational cost — or only for capital usage — will systematically misspecify the policy problem. The MILP formulation makes these trade-offs explicit and selectable.

6 Business Insights and Recommendations

Safety Stock Calibration. Per-SKU forecast error distributions derived from the LightGBM model enable safety stock levels to be calibrated to target service rates (e.g., 95% fill rate) with minimum excess inventory. This replaces fixed safety stock rules with empirically grounded, product-level protection that reflects each SKU's actual demand variability.

Category-Specific Replenishment Cycles. FOODS products benefit from daily replenishment driven by 7-day lag forecasts. HOBBIES products — intermittent and holiday-driven — are better served by weekly or bi-weekly replenishment tied to promotional calendars, reducing order complexity without sacrificing availability. HOUSEHOLD products warrant weekly cycles with promotional monitoring.

Pre-Event Stock Positioning. The model identifies 5–7 day pre-event demand escalation for major holidays. Operations teams should initiate stock pre-positioning 10 days before key events to avoid last-mile logistics bottlenecks during peak demand. Pre-holiday periods showed 2–4× normal sales volume in the M5 data, making early positioning especially critical for FOODS and HOUSEHOLD categories.

SNAP Day Planning. Dedicated SNAP benefit day forecasts should be incorporated into store-level ordering systems for high-velocity FOODS items. California stores exhibited the largest SNAP-driven uplift (up to 15%), and those locations should receive priority stock allocation during benefit windows.

Capital Allocation Under Budget Pressure. The portfolio model demonstrates that under binding capital constraints, inventory planning must be treated as a portfolio allocation problem rather than a collection of independent replenishment decisions. Managers should explicitly evaluate capital-to-cost ratios across SKUs and accept modest increases in total operating cost in exchange for improved capital efficiency when liquidity is constrained.

Supply Chain Efficiency. Aggregate store-level forecasts, where WRMSSE is substantially lower than at the individual product level, are immediately actionable for distribution center scheduling. Adoption of these forecasts is estimated to reduce stockout events by 12–18% and reduce overstock by 7–11% based on error distribution analysis.

7 Limitations

Several limitations of the present framework should be acknowledged. First, model development used a subset of the M5 dataset (200 products, 3 stores) rather than the full 3,049-product, 10-store dataset. This was necessary for computational feasibility but limits the generalizability of quantitative results to larger and more heterogeneous retail settings. Second, the simulation framework assumes fixed lead time. In practice, lead times vary due to supplier delays and logistics disruptions; stochastic lead times would increase simulation realism at the cost of additional parameter estimation. Third, demand uncertainty is modeled using a single coefficient of variation applied to forecast levels, which does not capture more complex demand behavior such as correlated shocks across products, regime shifts, or substitution effects. Fourth, the optimization objective is cost-based and does not explicitly incorporate product profit margins or strategic importance; a revenue-weighted formulation might assign different priorities to high-margin products. Fifth, the conversational AI assistant was evaluated functionally and qualitatively rather than through a formal user study measuring decision accuracy, response time, or user trust. Finally, the MILP portfolio model does not account for supplier capacity limits, minimum order quantities, or transportation constraints that would affect real-world policy feasibility.

8 Conclusion and Future Work

This project developed an end-to-end AI-driven inventory optimization framework that integrates demand forecasting, stochastic simulation, single-SKU replenishment optimization, MILP portfolio capital allocation, and a conversational AI decision assistant. The central finding is that forecasting alone is insufficient for effective inventory management: materially greater decision value is created when forecast outputs are connected to uncertainty quantification, simulation-based policy evaluation, and capital-constrained portfolio optimization. The integration of all five modules into a single coherent pipeline — with each module's outputs directly feeding the next — is the principal architectural contribution of this work.

LightGBM achieved the strongest forecasting performance among four candidate models, with holdout RMSE of 1.7933 and R^2 of 0.6502, benefiting from its native handling of categorical series identifiers and efficient leaf-wise tree growth. Simulation-based policy search recovered theoretically expected inventory trade-offs from empirical data. Demand shock testing demonstrated that policies appearing safe under baseline demand can require substantially greater capital commitment under +30–50% demand surges — a finding with direct practical implications for retail resilience planning. Portfolio optimization proved most valuable under binding capital constraints, where the MILP enforced cross-SKU trade-offs invisible to independent SKU-level analysis. The conversational AI layer improved accessibility by grounding natural-language responses in structured model outputs, reducing the interpretation burden placed on non-technical managers.

Future work will pursue six extensions. First, the framework will be extended to the full M5 dataset to validate scalability across 30,490 series. Second, probabilistic forecasting via quantile regression will replace point forecasts with full predictive distributions, enabling uncertainty-aware ordering rules. Third, stochastic lead times and supplier capacity constraints will be incorporated into both the simulation and MILP formulations. Fourth, the optimization objective will be extended to include margin weighting and product strategic priority. Fifth, Transformer-based sequence models (Temporal Fusion Transformer [5], TimeGPT) will be evaluated as forecasting alternatives. Sixth, the conversational AI assistant will be evaluated through a structured user study measuring decision accuracy and task completion time relative to a baseline dashboard interface.

9 Contributions

9.1 Team Member Contributions

This project was completed collaboratively by a three-member team. Responsibilities were allocated as follows to leverage individual expertise while ensuring cohesive integration of all analytical modules.

Kaiwalya Mishra led the data acquisition and preprocessing pipeline, including the reshaping of the M5 Walmart dataset from wide to long format, calendar and price table joins, missing price imputation, and the train/validation/holdout chronological split. He designed and implemented the full feature engineering framework—including lag-based demand features, rolling window statistics, EWMA features, calendar and event features, and price elasticity features—which collectively contributed 67% of total LightGBM model gain. Kaiwalya also conducted exploratory data analysis and produced the descriptive statistics and visualizations summarized in Section 3.

Aditya Chache was responsible for the core machine learning and optimization components. He implemented the global LightGBM forecasting pipeline, including candidate model training (Linear Regression, Random Forest, XGBoost, LightGBM), hyperparameter tuning, early stopping, and feature importance analysis. He developed the Monte Carlo stochastic simulation framework (Module 2) and the single-SKU (s, Q) policy search (Module 3). He also formulated and implemented the MILP multi-SKU portfolio optimization model (Module 4) using the PuLP solver, and conducted all demand shock and capital regime experiments reported in Section 5.

Dev Dhimant Patel designed and implemented the conversational AI decision assistant (Module 5), including the structured context memory schema, the grounded response generation architecture, and the multi-turn session management system. He integrated the outputs of Modules 1–4 into the conversational interface, implemented the three query-class handlers (analytical interpretation, simulation-oriented, and portfolio-level queries), and qualitatively evaluated assistant response quality. Dev also led the writing of Sections 4.5, 6 (Business Insights), and the Conclusion, and coordinated the final document assembly, formatting, and submission.

9.2 AI Tool Contributions

Generative AI tools were used to support—but not replace—the intellectual contributions of the team throughout this project. All analytical design decisions, model selections, experimental results, and interpretations represent the original work of the authors.

Claude, MCP, and Microsoft Copilot

The complete project codebase, including all data processing scripts, model training notebooks, simulation engine, MILP solver, and conversational assistant implementation, is publicly available at: https://github.com/adityachache/Agentic-AI-for-Inventory-Optimization

10 References

[1] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "M5 accuracy competition: Results, findings, and conclusions," Int. J. Forecasting, vol. 38, no. 4, pp. 1346–1364, 2022.

[2] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M4 Competition: 100,000 time series and 61 forecasting methods," Int. J. Forecasting, vol. 36, no. 1, pp. 54–74, 2020.

[3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma et al., "LightGBM: A highly efficient gradient boosting decision tree," in Advances in Neural Information Processing Systems, vol. 30, 2017.

[4] S. J. Taylor and B. Letham, "Forecasting at scale," Am. Statistician, vol. 72, no. 1, pp. 37–45, 2018.

[5] B. Lim, S. Ö. Arık, N. Loeff, and T. Pfister, "Temporal Fusion Transformers for interpretable multi-horizon time series forecasting," Int. J. Forecasting, vol. 37, no. 4, pp. 1748–1764, 2021.

[6] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. OTexts, 2021.

[7] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794.

[8] E. A. Silver, D. F. Pyke, and D. J. Thomas, Inventory and Production Management in Supply Chains, 4th ed. CRC Press, 2016.

[9] F. S. Hillier and G. J. Lieberman, Introduction to Operations Research, 11th ed. McGraw-Hill, 2021.

[10] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

11 Appendix

Appendix A: LightGBM Hyperparameter Configuration

Hyperparameter	Value	Rationale
num_leaves	128	Balance model complexity and overfitting risk
learning_rate	0.05	Slow learning rate; compensated by large tree count
n_estimators	2,000	With early stopping; patience = 200 rounds
min_child_samples	20	Regularization for sparse, intermittent series
subsample	0.80	Row-level stochastic boosting
colsample_bytree	0.80	Feature subsampling per tree
reg_alpha	0.10	L1 regularization coefficient
reg_lambda	0.10	L2 regularization coefficient
objective	regression	Mean squared error minimization
metric	rmse	Validation monitoring metric

Table A1: LightGBM Final Hyperparameter Configuration

Appendix B: Python Environment

Library	Version	Purpose
pandas	1.5.3	Data manipulation and feature engineering
numpy	1.24.2	Numerical computing
lightgbm	3.3.5	Global gradient boosting forecasting model
scikit-learn	1.2.2	Preprocessing, model evaluation, metrics
scipy / PuLP	1.10 / 2.7	Monte Carlo simulation; MILP optimization
statsmodels	0.14.0	ARIMA, ETS baseline models
matplotlib/seaborn	3.7.1 / 0.12	Data visualization and EDA
optuna	3.1.1	Bayesian hyperparameter optimization

Table A2: Python Environment and Key Dependencies

Appendix C: MILP Notation Summary

Symbol	Definition
i	SKU index
k	Policy index for SKU i

x_{ik}	Binary decision variable: 1 if policy k selected for SKU i
c_{ik}	Inventory management cost of SKU i under policy k
I_{ik}	Average inventory capital required: Ī_{ik} × unit_price_i
B	Total available inventory capital budget
s	Reorder point (units on hand triggering replenishment)
Q	Order quantity (units per replenishment event)
α	Minimum required fill rate (service-level threshold)
CV	Coefficient of variation for demand uncertainty in simulation

Table A3: MILP Model Notation

Appendix D: Dataset

Walmart M5 Dataset - https://www.kaggle.com/competitions/m5-forecasting-accuracy/data

Appendix E: Project Repository

The complete source code for this project—including all data preprocessing scripts, feature engineering pipelines, LightGBM training notebooks, Monte Carlo simulation engine, single-SKU optimization module, MILP portfolio solver, and conversational AI assistant—is publicly available on GitHub:

GitHub Repository: https://github.com/adityachache/Agentic-AI-for-Inventory-Optimization

The repository is organized into the following directories: data/ (raw and processed M5 dataset files), src/ (Python source modules for the simulation and optimization pipeline), assistant/ (conversational AI assistant implementation), and results/ (saved model artifacts, simulation outputs, and optimization results). A README.md file at the repository root provides setup instructions, dependency requirements, and a quickstart guide for reproducing all reported results.

— End of Report —

Capstone Project | College of Business and Economics, California State University, East Bay | Academic Year 2025–2026